Probe Station — Equation Reference

Probe Station · generated from source · 2026-07-27

What this document is: the complete, authoritative list of every equation the simulator evaluates, where each one lives in the code, which control-panel parameters feed it, and which on-screen output it produces.

Every equation is **declared exactly once**, in a formula registry, as a pure function plus its KaTeX, its teaching concept, and its assumptions. Nothing in the UI, the charts, or the AI tutor re-implements physics — they all read the same declared formulas. When a formula is evaluated it returns *both* the number and the derivation that produced it, from the same call, so the explanation shown to the student can never drift from the math.

There are **14 declared closed-form equations** and **9 numerical procedures**. A single parameter change re-runs the whole chain in a fixed order: device physics → per-transistor current → circuit solve (iterative) → circuit metrics. Every output on screen comes from its own equation.

For a CMOS inverter at default settings, one slider move triggers roughly **25,000 evaluations of the drain-current law alone** (201-point transfer sweep × 48 bisection iterations × 2 transistors, plus the operating point, leakage vectors and switching-threshold search). That is why there cannot be one equation — the tool solves a circuit, not a formula.

---

1. Notation and constants

1.1 Physical constants

Declared once in `src/domain/primitives/mosfet/constants.ts`. No formula redefines these.

Symbol	Name	Value	Unit
`q`	Elementary charge	$1.602176634 \times 10^{-19}$	C
`k`	Boltzmann constant	1.380649×10^{-23}	J/K
`ε₀`	Vacuum permittivity	$8.8541878128 \times 10^{-12}$	F/m
`εr,Si`	Relative permittivity of silicon	11.7	—
`εr,ox`	Relative permittivity of SiO_2	3.9	—
`ε_si`	Permittivity of silicon	$\varepsilon_{r,Si} \cdot \varepsilon_0$	F/m
`ε_ox`	Permittivity of gate oxide	$\varepsilon_{r,ox} \cdot \varepsilon_0$	F/m
`n_i`	Intrinsic carrier concentration (Si, 300 K)	1.0×10^{16}	m^{-3}

1.2 Model constants

Symbol	Name	Value	Unit
`T₀`	Reference temperature	300	K
`α`	Threshold temperature coefficient	-2.0×10^{-3}	V/K
—	Mobility temperature exponent	−1.5	—

1.3 Per-device-type constants

Not user-editable; they are what make PMOS intrinsically weaker than NMOS (the canonical reason PMOS is sized wider). Source: `src/domain/devices/shared.ts`.

Symbol	NMOS	PMOS	Unit
μ_0 (low-field mobility)	0.045 (~450 cm²/V·s, electrons)	0.020 (~200 cm²/V·s, holes)	m²/V·s
λ (channel-length modulation)	0.05	0.05	1/V
n (subthreshold slope factor)	1.3	1.3	—

1.4 Process-corner adjustment

`src/domain/primitives/mosfet/corner.ts`. A corner is not an equation — it is a pair of offsets injected into equations **E4** (threshold) and **E5** (mobility).

Corner	ΔV_corner (V)	s_corner (mobility scale)
TT — Typical	0	1.0
FF — Fast	–0.05	1.1
SS — Slow	+0.05	0.9
FS — Fast N / Slow P	NMOS: –0.05, PMOS: +0.05	NMOS: 1.1, PMOS: 0.9
SF — Slow N / Fast P	NMOS: +0.05, PMOS: –0.05	NMOS: 0.9, PMOS: 1.1

---

2. Control-panel parameters (the inputs)

2.1 Gate devices — CMOS Inverter, NAND

Schema: `standardCmosSchema` in `src/domain/devices/shared.ts`.

Group	Control	Symbol	Range	Default	Unit
Geometry	Channel Length	`L`	20 n – 1 μ	180 n	m
Geometry	Gate Width	`W`	50 n – 5 μ	1 μ	m
Geometry	Oxide Thickness	`T_ox`	1 n – 20 n	4 n	m
Process	Channel Doping	`N_a`	1×10^{21} – 1×10^{24} (log)	1×10^{23}	m^{-3}
Process	Threshold Voltage	`V_th0`	0.2 – 0.8	0.4	V
Process	Process Corner	`corner`	TT / FF / SS / FS / SF	TT	—
Operating	Supply Voltage	`V_DD`	0.4 – 3.3	1.8	V
Operating	Input Voltage	`V_in`	0 – 3.3	0.9	V
Operating	Load Capacitance	`C_L`	1 f – 1 p	10 f	F
Operating	Temperature	`T`	233 – 423	300	K

2.2 Single-transistor explorer — NMOS, PMOS

Schema: `transistorSchema` in `src/domain/devices/transistor-shared.ts`. Doping, nominal threshold and corner are held at internal defaults (`N_a` = 1×10^{23} m^{-3}, `V_th0` = 0.4 V, corner = TT) because this view teaches device behaviour, not process selection.

Group	Control	Symbol	Range	Default	Unit
Bias	Gate–Source Voltage	`V_GS`	0 – 1.8	1.0	V
Bias	Drain–Source Voltage	`V_DS`	0 – 1.8	1.0	V
Geometry	`W`, `L`, `T_ox`	—	as above	as above	m
Environment	Temperature	`T`	233 – 423	300	K

3. Layer 1 — Device physics equations

All nine are declared in `src/domain/primitives/mosfet/formulas.ts`, each as a `defineFormula` block carrying its LaTeX, concept ID and assumptions.

E1 · Thermal voltage

```
V_T = k · T / q
```

- **Inputs:** `T` | **Output:** V | **Code:** `formulas.ts:14`
- Sets the scale of carrier statistics. Feeds **E3** and **E9**.

E2 · Oxide capacitance (per unit area)

```
C_ox = ε_ox / T_ox
```

- **Inputs:** `T_ox` | **Output:** F/m² | **Code:** `formulas.ts:24`
- Thinner oxide → larger gate capacitance → stronger drive. Feeds **E4** and **E6**.
- *Assumption:* ε_ox = 3.9 · ε_0 (SiO_2).

E3 · Bulk (Fermi) potential

```
φ_F = V_T · ln( N_a / n_i )
```

- **Inputs:** `V_T`, `N_a` | **Output:** V | **Code:** `formulas.ts:34`
- *Assumption:* n_i taken at 300 K (1.0×10^{16} m^{-3}).

E4 · Effective threshold voltage

```
V_th = V_th0 + γ · ( √(2φ_F + V_SB) − √(2φ_F) ) + α · (T − T₀) + ΔV_corner

  where   γ = √(2 · q · ε_si · N_a) / C_ox        (body-effect factor)
```

- **Inputs:** `V_th0`, `C_ox`, `N_a`, `φ_F`, `V_SB`, `T`, `ΔV_corner` | **Output:** V
- **Code:** `formulas.ts:50` · assembled in `threshold.ts:12`
- Four independent physical effects in one expression: nominal threshold, body bias, temperature drift, process spread. γ is computed internally rather than exposed as a control.
- *Assumption:* α = -2.0×10^{-3} V/K.

E5 · Carrier mobility

```
μ = μ₀ · (T / T₀)^(−1.5) · s_corner
```

- **Inputs:** `μ₀`, `T`, `s_corner` | **Output:** m²/V·s | **Code:** `formulas.ts:81`
- Mobility falls as temperature rises (phonon scattering). This is why a hot chip is a slow chip.
- *Assumption:* phonon-limited $\mu(T) \propto T^{-1.5}$.

E6 · Process transconductance

```
k′ = μ · C_ox
```

- **Inputs:** `μ`, `C_ox` | **Output:** A/V² | **Code:** `formulas.ts:100`
- The single number that carries all process strength into the current laws **E7–E9**.

R1 · Region-of-operation selection

Not a formula but the branch that chooses between **E7**, **E8** and **E9**. Implemented in `mosfet.model.ts`.

```
V_ov = V_GS − V_th                    (gate overdrive)

  V_ov ≤ 0        → cutoff       → use E9 (subthreshold)
  V_DS <  V_ov    → triode       → use E8
  V_DS ≥  V_ov    → saturation   → use E7
```

PMOS is handled by passing **source-referenced magnitudes**, so the same three equations serve both device types — there is no duplicated PMOS current law.

E7 · Drain current — saturation

```
I_D = ½ · k′ · (W / L) · V_ov² · (1 + λ · V_DS)
```

- **Inputs:** `k′`, `W`, `L`, `V_ov`, `λ`, `V_DS` | **Output:** A | **Code:** `formulas.ts:111`
- Current is set by gate overdrive and is nearly flat in V_DS.
- *Assumption:* long-channel square law; λ models channel-length modulation.

E8 · Drain current — triode (linear)

```
I_D = k′ · (W / L) · ( V_ov · V_DS − V_DS² / 2 ) · (1 + λ · V_DS)
```

- **Inputs:** `k′`, `W`, `L`, `V_ov`, `V_DS`, `λ` | **Output:** A | **Code:** `formulas.ts:132`
- The channel behaves as a voltage-controlled resistor.

E9 · Drain current — subthreshold (leakage)

```
I_D = k′ · (W / L) · (n − 1) · V_T² · e^((V_GS − V_th) / (n · V_T)) · ( 1 − e^(−V_DS / V_T) )
```

- **Inputs:** `k′`, `W`, `L`, `n`, `V_T`, `V_GS`, `V_th`, `V_DS` | **Output:** A
- **Code:** `formulas.ts:155` · applied via `leakage.ts:13`
- Exponential in gate voltage. At V_GS = 0 this *is* the off-state leakage I_off that sets static power and rises sharply with temperature.
- *Assumption:* weak-inversion (EKV-style) approximation.

---

4. Layer 2 — Circuit solve (numerical, no closed form)

These produce values the UI labels as **"found numerically"**. They are the reason the outputs are not a single algebraic expression: an inverter's output voltage has no closed form once both transistors are in arbitrary regions, so it is solved by bisection.

Source: `network-solver.ts`, `analytical.engine.ts`.

N1 · Network branch current (recursive)

```
parallel branch:   I = Σ I_child(V_top, V_bottom)
device:            I = E7 / E8 / E9  as selected by R1
series branch:     see N2
```

- **Code:** `network-solver.ts:26`
- Topology-agnostic: the inverter, NAND, NOR and AOI gates are all solved by this same code — only the netlist tree differs.

N2 · Series-chain internal node voltage

```
find V_mid  such that   I_head(V_top, V_mid) = I_tail(V_mid, V_bottom)
method: bisection, 48 iterations
```

- **Code:** `network-solver.ts:83`
- Needed for stacked devices (e.g. the series NMOS pull-down of a NAND).

N3 · Output operating point — KCL at the output node

```
find V_out  such that   I_pullup(V_DD → V_out) = I_pulldown(V_out → 0)
method: bisection over [0, V_DD], 48 iterations

reported through-current:   I = ( I_pulldown + I_pullup ) / 2
```

- **Code:** `network-solver.ts:161`
- **Outputs:** *Output Voltage (V_out)* and *Through / Short-Circuit Current*.

N4 · Voltage transfer characteristic (VTC) sweep

```
V_in,i = V_DD · i / (N − 1),     i = 0 … N−1,     N = 201 (default)
for each i:  solve N3  →  (V_in, V_out, I)
```

- **Code:** `analytical.engine.ts:60`
- **Outputs:** the *VTC* chart and the *Short-Circuit Current* chart.
- Monte Carlo runs use N = 2, since only the scalar metrics are needed.

N5 · Switching threshold V_M

```
find V_in  such that   V_out(V_in) = V_in
method: bisection over [0, V_DD], 60 iterations (each iteration re-solves N3)
```

- **Code:** `analytical.engine.ts:186`
- **Output:** *Switching Threshold (V_M)* — the high-gain trip point.

N6 · Worst-case static leakage

```
I_leak = max over all characteristic input vectors of  N3.current

  inverter: vectors { IN=0 }, { IN=1 }
  NAND:     vectors { A,B } ∈ {00, 01, 10, 11}
```

- **Code:** `analytical.engine.ts:122`
- **Output:** *Leakage.* Feeds **E13** (static power).

N7 · On-currents for delay

```
I_on,pulldown = N1( pull-down network, V_top = V_DD, V_bottom = 0, all inputs HIGH )
I_on,pullup   = N1( pull-up   network, V_top = V_DD, V_bottom = 0, all inputs LOW  )
```

- **Code:** `analytical.engine.ts:139`
- Feeds **E10**.

N8 · Transconductance g_m (single-transistor view)

```
g_m = ∂I_D / ∂V_GS  ≈  ( I_D(V_GS + Δ) − I_D(V_GS − Δ) ) / (2Δ),     Δ = 0.01 V
```

- **Code:** `transistor.engine.ts:66`
- **Output:** *Transconductance* in the NMOS/PMOS explorer.
- Deliberately a central difference on the *same* current law rather than a second algebraic formula — so g_m can never disagree with the plotted I_D. Δ is clamped to the slider range.

N9 · Single-transistor I–V families

```
I_D–V_DS family:  5 curves at V_GS = V_GS,max · i / 5,  i = 1…5;  56 points each
I_D–V_GS curve:   56 points, V_GS = V_GS,max · i / 55,  at the user's V_DS
```

- **Code:** `transistor.engine.ts:35`

---

5. Layer 3 — Circuit metric equations

Declared in `metrics.formulas.ts`, same registry mechanism as the device equations.

E10 · Propagation delay, one edge

```
t_p = C_L · V_DD / ( 2 · I_on )
```

- **Inputs:** `C_L`, `V_DD`, `I_on` (from **N7**) | **Output:** s | **Code:** `metrics.formulas.ts:6`
- Evaluated **twice**: `t_pHL` with I_on,pulldown, `t_pLH` with I_on,pullup.
- I_on is floored at 1×10^{-18} A to avoid division by zero.
- *Assumption:* average-current approximation, not a full transient solve.

E11 · Average propagation delay

```
t_p = ( t_pHL + t_pLH ) / 2
```

- **Code:** `metrics.formulas.ts:18`
- **Output:** *Propagation Delay*.

E12 · Dynamic power

```
P_dyn = α · C_L · V_DD² · f

  with  α = 1  (activity factor)
        f = 1 / (2 · t_p)   — the max toggle rate implied by the delay above
```

- **Code:** `metrics.formulas.ts:29`, frequency derived at `analytical.engine.ts:161`
- **Output:** *Dynamic Power*. This is the V_{DD}^2 term that makes supply scaling so powerful.

E13 · Static power

```
P_stat = I_leak · V_DD
```

- **Inputs:** `I_leak` (from **N6**), `V_DD` | **Code:** `metrics.formulas.ts:41`
- **Output:** *Static Power*.

E14 · Total power

```
P = P_dyn + P_stat
```

- **Code:** `metrics.formulas.ts:52`
- **Output:** *Total Power*.

---

6. Layer 4 — Variation, statistics and analysis

S1 · Monte Carlo process sampling

`montecarlo.ts:23`

```
V_th0 ~ N( V_th0,  σ = max(0.02, 0.05 · V_th0) )   clamped to [0.15, 0.85] V
L     ~ N( L,      σ = 0.06 · L )                  clamped to [20 nm, 1 µm]
T_ox  ~ N( T_ox,   σ = 0.03 · T_ox )               clamped to [1 nm, 20 nm]
```

Each sample re-runs the **entire** chain above (E1–E14, N1–N7) with N = 2 sweep points, and records propagation delay, leakage and switching threshold. The corner sets the mean; this adds the spread.

S2 · Pseudo-random number generator — mulberry32

`montecarlo.ts:63` — deterministic and seeded, so every Monte Carlo run is reproducible and testable. `Math.random` is never used.

S3 · Gaussian sampling — Box–Muller transform

```
z = √( −2 · ln u₁ ) · cos( 2π · u₂ ),     u₁, u₂ ~ Uniform(0,1)
```

- **Code:** `montecarlo.ts:75`

S4 · Histogram statistics

`histogram.ts:21`

```
mean  = ( Σ vᵢ ) / N
std   = √( Σ (vᵢ − mean)² / N )          (population standard deviation)
width = ( max − min ) / binCount,        binCount = 24
bin index of v = min( binCount − 1, ⌊ (v − min) / width ⌋ )
```

S5 · Yield fraction

```
yield = count( v ≤ limit )  / N      ("below" spec)
      = count( v ≥ limit )  / N      ("above" spec)
```

- **Code:** `histogram.ts:50`

S6 · Impact / sensitivity percentages

```
Δ%        = ( to − from ) / |from| × 100
W/L ratio = W / L
```

- **Code:** `impact.ts:51`

- Reported for k′, V_th, W/L, propagation delay, leakage and total power. Lines below 0.5 % change are suppressed as noise. All directions are *measured* from before/after engine results — no physics is invented in the narrative layer.

S7 · VTC region boundaries (the A–E bands on the chart)

`builders.ts` — annotation geometry, derived from measured values:

```
A|B boundary:  V_in = V_tn                (NMOS turns on)      — exact
D|E boundary:  V_in = V_DD − |V_tp|       (PMOS turns off)     — exact
region C:      centred on V_M, half-width = max( sweep step / 2, (b_DE − b_AB) · 0.05 )
```

Region C is drawn with a small finite width for legibility; in the ideal square-law model with λ → 0 it would be vertical.

Band	Condition	State
A	V_in < V_tn	NMOS cutoff · PMOS linear (V_out = V_DD)
B	V_tn < V_in < ~V_M	NMOS saturation · PMOS linear
C	V_in ≈ V_M	NMOS saturation · PMOS saturation (steep transition)
D	~V_M < V_in < V_DD – |V_tp|	NMOS linear · PMOS saturation
E	V_in > V_DD – |V_tp|	NMOS linear · PMOS cutoff (V_out = 0)

7. Evaluation order — what happens on one parameter change

```
CONTROL PANEL CHANGE
        │
        ▼
  buildMosfetParams()  →  per-transistor SI parameter set (NMOS and PMOS separately)
        │
        ▼
  ── LAYER 1 · per transistor ──────────────────────────────────
  E1  V_T        ← T
  E2  C_ox       ← T_ox
  E3  φ_F        ← V_T, N_a
  E4  V_th       ← V_th0, C_ox, N_a, φ_F, V_SB, T, corner
  E5  μ          ← μ₀, T, corner
  E6  k′         ← μ, C_ox
  R1  region     ← V_GS, V_th, V_DS
  E7/E8/E9  I_D  ← k′, W, L, V_ov, V_DS, λ  (or n, V_T for subthreshold)
        │
        ▼
  ── LAYER 2 · circuit solve (iterative, calls Layer 1 thousands of times) ──
  N1/N2  branch currents over the netlist tree
  N3     V_out and through-current at the user's V_in      → Output Voltage, Current
  N4     201-point VTC sweep                               → VTC chart, I_SC chart
  N5     V_M                                               → Switching Threshold
  N6     worst-case leakage                                → Leakage
  N7     on-currents
        │
        ▼
  ── LAYER 3 · metrics ─────────────────────────────────────────
  E10 t_pHL, t_pLH   ← C_L, V_DD, I_on
  E11 t_p            ← t_pHL, t_pLH                        → Propagation Delay
  E12 P_dyn          ← C_L, V_DD, f(t_p)                   → Dynamic Power
  E13 P_stat         ← I_leak, V_DD                        → Static Power
  E14 P_total        ← P_dyn, P_stat                       → Total Power
        │
        ▼
  ── LAYER 4 · optional ────────────────────────────────────────
  S1–S5  Monte Carlo distributions and yield
  S6     before/after impact deltas
```

8. Which control affects which equation

● = direct input to that equation. ○ = affects it indirectly, through an upstream result.

Control	E1 V_T	E2 C_ox	E3 ϕ_F	E4 V_th	E5 μ	E6 k′	E7/E8 I_D	E9 I_leak	E10/E11 t_p	E12 P_dyn	E13 P_stat
`L`							•	•	○	○	○
`W`							•	•	○	○	○
`T_ox`		•		○		○	○	○	○	○	○
`N_a`			•	•			○	○	○	○	○
`V_th0`				•			○	○	○	○	○
`corner`				•	•	○	○	○	○	○	○
`V_DD`							○	○	•	•	•
`V_in`							•	•			
`C_L`									•	•	
`T`	•		○	•	•	○	○	○	○	○	○

Note how `T` and `corner` touch almost everything — that is the intended teaching point, and it is why the tool cannot collapse to a single equation.

9. Output → equation map (quick lookup)

On-screen output	Produced by	Type
Output Voltage `V_out`	**N3** (KCL bisection)	numerical
Through / Short-Circuit Current	**N3**	numerical
Per-transistor Region	**R1**	branch
Per-transistor Drain Current	**E7 / E8 / E9**	closed form
Per-transistor Threshold `V_th`	**E4** (← E1, E2, E3)	closed form
Per-transistor Overdrive `V_ov`	**R1**	closed form
Transconductance `g_m`	**N8**	numerical
Switching Threshold `V_M`	**N5**	numerical
Leakage	**N6** (← E9)	numerical
Propagation Delay	**E11** (← E10 ← N7)	closed form
Dynamic Power	**E12**	closed form
Static Power	**E13** (← N6)	closed form
Total Power	**E14**	closed form
VTC chart	**N4**	numerical sweep
Short-Circuit Current chart	**N4**	numerical sweep
VTC region bands A–E	**S7**	derived annotation
I_D–V_DS family	**N9**	numerical sweep
I_D–V_GS curve	**N9**	numerical sweep
Monte Carlo histograms	**S1–S4**	statistical
Yield %	**S5**	statistical
Impact card deltas	**S6**	measured

Appendix A · LaTeX source, as declared in code

Copy-paste ready for slides or a paper. These strings are the literal `latex:` fields in the formula registry — the same strings the app renders to the student.

ID	LaTeX
E1 `thermal-voltage`	`V_T = \dfrac{k T}{q}`
E2 `oxide-capacitance`	`C_{ox} = \dfrac{\varepsilon_{ox}}{T_{ox}}`
E3 `bulk-potential`	`\phi_F = V_T \ln\!\left(\dfrac{N_a}{n_i}\right)`
E4 `threshold-voltage`	`V_{th} = V_{th0} + \gamma\left(\sqrt{2\phi_F + V_{SB}} - \sqrt{2\phi_F}\right) + \alpha\,(T - T_0) + \Delta V_{corner}`
E5 `carrier-mobility`	`\mu = \mu_0 \left(\dfrac{T}{T_0}\right)^{-1.5} \cdot s_{corner}`
E6 `process-transconductance`	`k' = \mu\,C_{ox}`
E7 `drain-current-saturation`	`I_D = \tfrac{1}{2} k' \dfrac{W}{L} V_{ov}^2 (1 + \lambda V_{DS})`
E8 `drain-current-triode`	`I_D = k' \dfrac{W}{L}\left(V_{ov} V_{DS} - \tfrac{V_{DS}^2}{2}\right)(1 + \lambda V_{DS})`
E9 `drain-current-subthreshold`	`I_D = k' \dfrac{W}{L}(n-1) V_T^2\, e^{(V_{GS}-V_{th})/n V_T}\left(1 - e^{-V_{DS}/V_T}\right)`
E10 `propagation-delay-half`	`t_{p} = \dfrac{C_L\,V_{DD}}{2\,I_{on}}`
E11 `propagation-delay-average`	`t_p = \dfrac{t_{pHL} + t_{pLH}}{2}`
E12 `dynamic-power`	`P_{dyn} = \alpha\,C_L\,V_{DD}^2\,f`
E13 `static-power`	`P_{stat} = I_{leak}\,V_{DD}`
E14 `total-power`	`P = P_{dyn} + P_{stat}`
N8 `transconductance`	`g_m = \frac{\partial I_D}{\partial V_{GS}} \approx \frac{I_D(V_{GS}+\Delta) - I_D(V_{GS}-\Delta)}{2\Delta}`

Appendix B · Modelling scope and limitations

Stated plainly, because these bound what the tool should be used to teach:

1. **Long-channel square law.** E7/E8 are the classical square-law equations. Velocity saturation, DIBL and quantum effects are not modelled; below ~45 nm the absolute numbers drift from silicon even though the trends stay correct.
2. **Delay is an average-current approximation** (E10), not a transient solve. It captures the C·V/I scaling that matters pedagogically, not SPICE-accurate edge shapes.
3. **Activity factor α is fixed at 1** in E12, and frequency is derived as 1/(2·t_p) — i.e. the circuit toggling as fast as its own delay allows. Both are teaching simplifications.
4. **Monte Carlo is a teaching model, not signoff.** Three varied parameters, Gaussian, uncorrelated.
5. **Leakage is subthreshold only.** Gate tunnelling and junction leakage are not included.
6. **Body effect uses V_SB = 0** for the standalone transistor explorer; the gate solver supplies the real source-body bias per device.
7. **No parasitics.** Interconnect R/C and self-loading are outside the model; C_L is the single lumped load.

Generated from source. Every equation above is declared exactly once in the codebase — this document is a reading of the registry, not a parallel copy. If a formula changes in code, this file must be regenerated.